

Simon Chen

347-322-4133 | shangminch@gmail.com | simon-chen.com | linkedin.com/in/shangmin-chen | github.com/shangminchen

EDUCATION

Boston University

Boston, MA

Bachelor of Arts in Computer Science

Graduated: Spring 2026

- Relevant Coursework: Data Structures & Algorithms, Operating Systems, Distributed Systems, Databases, Machine Learning, Data Science.

EXPERIENCE

Software Engineering Intern

February 2026 – June 2026

RESET Standard

Shanghai, China

- Doubled the platform's supported environmental data categories (from 2 to 4) by independently implementing end-to-end water and waste data integration, developing Rails backend ingestion pipelines, PostgreSQL schemas, and client-facing React/Vite/Tailwind dashboards.
- Engineered asynchronous data pipelines with RabbitMQ, Redis, and Sidekiq to process long-running ingestion jobs in the background, decoupling external API calls from user-facing requests.
- Redesigned project-device mapping workflows in response to client requests, refactoring legacy forms to streamline onboarding of environmental monitoring devices.

Product Lead

December 2025 – Present

Ezesports

New York, NY

- Migrated an esports league platform from Google Sites to Next.js with a Supabase backend, replacing manual content updates with live scoring and automated season management.
- Designed a PostgreSQL schema on Supabase using Drizzle ORM, supporting 500+ matches across 20+ teams, and built RESTful endpoints for player statistics, standings, and historical data.

PROJECTS

Persephone | *Python, Cython/C++17, Rust, Modal, FAISS*

May 2026

- Built and operated a live Kalshi BTC prediction-market trading system end to end (market-data ingestion, theoServing, strategy evaluation, execution, and risk), generating \$8K net PnL on \$200K traded volume in its first 4 days.
- Replaced KDE inference with a GBM theoS layer, cutting model query latency from $\sim 1.5\text{ms}$ to $\sim 20\mu\text{s}$ per query ($\sim 75\times$) behind an unchanged live serving interface.
- Rewrote 9 live hot-path kernels (order-book maintenance, order-book imbalance, trade intensity, event aggregation) as native Cython extensions, cutting Kalshi book maintenance from 21.8ms to $195\mu\text{s}$ p50 ($112\times$).
- Designed a lock-free SPSC event ring in Cython/C++17 with cache-line-aligned atomic cursors and packed uint64 records, benchmarked at 584ns per enqueue and 703ns per event drained.

Hermes Letters | *Next.js, TypeScript, Supabase, Drizzle ORM, PostgreSQL*

June 2026

- Built a private long-form messaging platform for a group of 20+ friends during a study abroad semester, delivering 30+ letters.
- Ensured consistent letter claims via atomic SQL updates and secured sessions in XSS-resistant httpOnly cookies.
- Derived user connections at query time from kept-letter records using indexed lookups, avoiding a separate relationship table and associated database sync issues.
- Hardened media storage by validating uploads via server-side magic-byte signatures, storing files in a private Supabase bucket, and restricting delivery to 1-hour signed URLs.

TECHNICAL SKILLS

Languages: Python, C++17, Cython, Rust, TypeScript, JavaScript, Java, Ruby, SQL

Frameworks & Libraries: React, Next.js, Node.js, React Native, Ruby on Rails, Django, Spring Boot, FastAPI, Tailwind CSS, FAISS, CuPy

Databases & Infrastructure: PostgreSQL, SQLite, Redis, RabbitMQ, Sidekiq, Supabase, Docker, Nginx, Cloudflare, Modal, OpenMP